



Lecture # 24-25 Concurrency Control

Rashmi Dutta Baruah

Department of Computer Science & Engineering



भारतीय प्रौद्योगिकी संस्थान गुवाहाटी
Indian Institute of Technology Guwahati

Guwahati - 781039, INDIA





Outline

- Lock-Based Protocols
- Timestamp-Based Protocols
- Snapshot Isolation



Lock-Based Protocols

- A lock is a mechanism to control concurrent access to a data item
- Data items can be locked in two modes :
 1. **exclusive** (*X*) *mode*. Data item can be both read as well as written. X-lock is requested using **lock-X** instruction.
 2. **shared** (*S*) *mode*. Data item can only be read. S-lock is requested using **lock-S** instruction.
- Lock requests are made to concurrency-control manager. Transaction can proceed only after request is granted.



Lock-Based Protocols (Cont.)

- **Lock-compatibility matrix**

	S	X
S	true	false
X	false	false

- A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions
- Any number of transactions can hold shared locks on an item,
- But if any transaction holds an exclusive lock on the item no other transaction may hold any lock on the item.



Lock-Based Protocols (Cont.)

- Example of a transaction performing locking:

T_2 : **lock-S(A);**
read (A);
unlock(A);

lock-S(B);
read (B);
unlock(B);
display(A+B)

- Locking as above is not sufficient to guarantee serializability



Schedule With Lock Grants

- Grants omitted in later schedules
 - Assume grant happens just before the next instruction following lock request
- This schedule is not serializable
- A **locking protocol** is a set of rules followed by all transactions while requesting and releasing locks.
- Locking protocols enforce serializability by restricting the set of possible schedules.

T_1	T_2	concurrency-control manager
lock-X(B)		grant-X(B, T_1)
read(B)		
$B := B - 50$		
write(B)		
unlock(B)		
	lock-S(A)	
		grant-S(A, T_2)
	read(A)	
	unlock(A)	
	lock-S(B)	
		grant-S(B, T_2)
	read(B)	
	unlock(B)	
	display($A + B$)	
lock-X(A)		grant-X(A, T_1)
read(A)		
$A := A + 50$		
write(A)		
unlock(A)		



Deadlock

- Consider the partial schedule

T_3	T_4
lock-X(B)	
read(B)	
$B := B - 50$	
write(B)	
	lock-S(A)
	read(A)
	lock-S(B)
lock-X(A)	

- Neither T_3 nor T_4 can make progress — executing **lock-S(B)** causes T_4 to wait for T_3 to release its lock on B , while executing **lock-X(A)** causes T_3 to wait for T_4 to release its lock on A .
- Such a situation is called a **deadlock**.
 - To handle a deadlock one of T_3 or T_4 must be rolled back and its locks released.

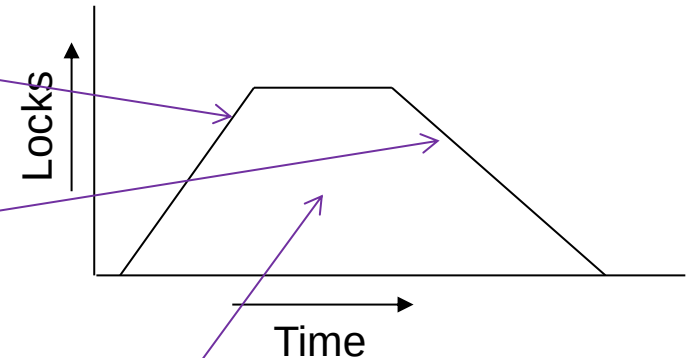


Deadlock (Cont.)

- The potential for deadlock exists in most locking protocols. Deadlocks are a necessary evil.
- **Starvation** is also possible if concurrency control manager is badly designed. For example:
 - A transaction may be waiting for an X-lock on an item, while a sequence of other transactions request and are granted an S-lock on the same item.
 - The same transaction is repeatedly rolled back due to deadlocks.
- Concurrency control manager can be designed to prevent starvation.

The Two-Phase Locking Protocol

- A protocol which ensures conflict-serializable schedules.
- Phase 1: **Growing Phase**
 - Transaction may obtain locks
 - Transaction may not release locks
- Phase 2: **Shrinking Phase**
 - Transaction may release locks
 - Transaction may not obtain locks



The protocol assures serializability. The transactions can be serialized in the order of their **lock points** (i.e., the point where a transaction acquired its final lock).



T5	T6	T7
lock-X(<i>A</i>) read(<i>A</i>) lock-S(<i>B</i>) read(<i>B</i>) write(<i>A</i>) unlock(<i>A</i>)	lock-X(<i>A</i>) read(<i>A</i>) write(<i>A</i>) unlock(<i>A</i>)	lock-S(<i>A</i>) read(<i>A</i>)

Partial schedule under two-phase locking (not cascadeless)



The Two-Phase Locking Protocol (Cont.)

- Two-phase locking *does not* ensure freedom from deadlocks
- Extensions to basic two-phase locking needed to ensure recoverability of freedom from cascading roll-back
 - **Strict two-phase locking:** a transaction must hold **all its exclusive locks** till it commits/aborts.
 - Ensures recoverability and avoids cascading roll-backs
 - **Rigorous two-phase locking:** a transaction must hold **all** locks till commit/abort.
 - Transactions can be serialized in the order in which they commit.
- Most databases implement rigorous two-phase locking, *but refer to it as simply two-phase locking*



Locking Protocols

- Given a locking protocol (such as 2PL)
 - A schedule S is **legal** under a locking protocol if it can be generated by a set of transactions that follow the protocol
 - A protocol **ensures** serializability if all legal schedules under that protocol are serializable



Lock Conversions

- Two-phase locking protocol with lock conversions:
 - Growing Phase:
 - can acquire a lock-S on item
 - can acquire a lock-X on item
 - can **convert** a lock-S to a lock-X (**upgrade**)
 - Shrinking Phase:
 - can release a lock-S
 - can release a lock-X
 - can convert a lock-X to a lock-S (**downgrade**)
- This protocol ensures serializability



Lock Conversions

T8: read($a1$);
read($a2$);
...
read(a_n);
write($a1$).

T9: read($a1$);
read($a2$);
display($a1 + a2$).

T8
lock-S($a1$)

lock-S($a2$)

lock-S($a3$)

lock-S($a4$)

lock-S(a_n)

upgrade($a1$)

T9

lock-S($a1$)

lock-S($a2$)

unlock($a1$)

unlock($a2$)



Automatic Acquisition of Locks

- A transaction T_i issues the standard read/write instruction, without explicit locking calls.
- The operation **read**(D) is processed as:
 - if** T_i has a lock on D
 - then**
 - $\text{read}(D)$
 - else begin**
 - if necessary wait until no other transaction has a **lock-X** on D
 - grant T_i a **lock-S** on D ;
 - $\text{read}(D)$
 - end**



Automatic Acquisition of Locks (Cont.)

- The operation **write**(D) is processed as:
 if T_i has a **lock-X** on D
 then
 write(D)
 else begin
 if necessary wait until no other trans. has any
lock on D ,
 if T_i has a **lock-S** on D
 then
 upgrade lock on D to **lock-X**
 else
 grant T_i a **lock-X** on D
 write(D)
 end;
- **All locks are released after commit or abort**

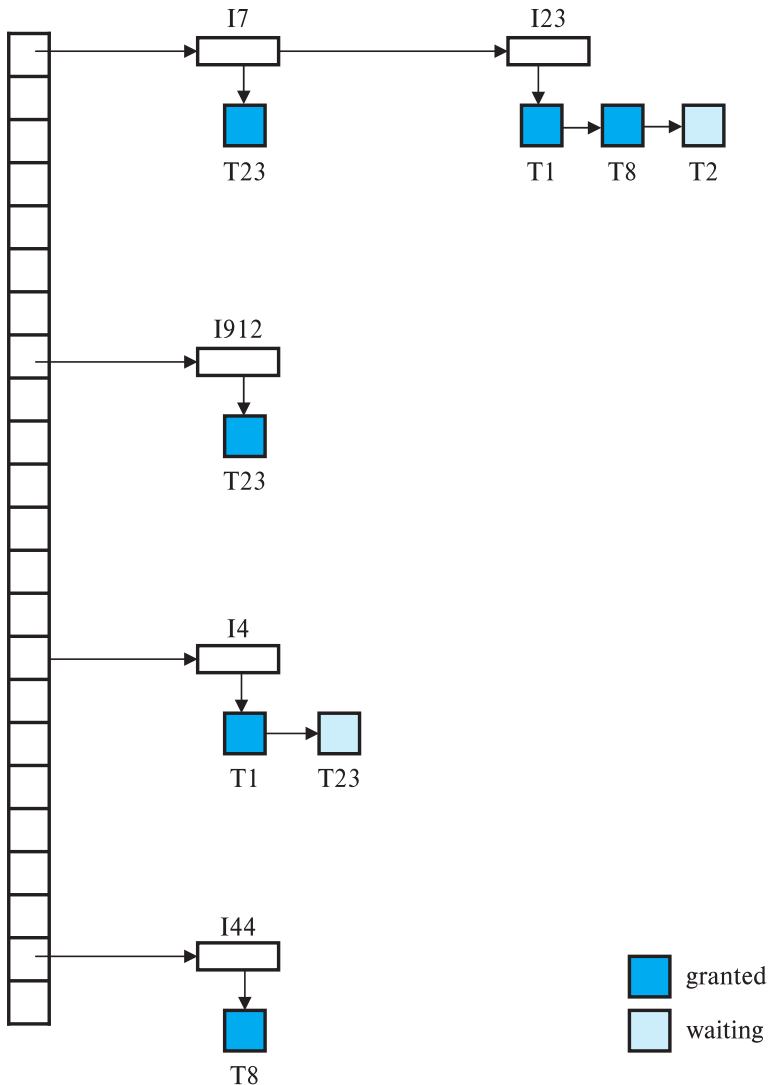


Implementation of Locking

- A **lock manager** can be implemented as a separate process
- Transactions can send lock and unlock requests as messages
- The lock manager replies to a lock request by sending a lock grant messages (or a message asking the transaction to roll back, in case of a deadlock)
 - The requesting transaction waits until its request is answered
- The lock manager maintains an in-memory data-structure called a **lock table** to record granted locks and pending requests

Lock Table

- Dark rectangles indicate granted locks, light colored ones indicate waiting requests
- Lock table also records the type of lock granted or requested
- New request is added to the end of the queue of requests for the data item, and granted if it is compatible with all earlier locks
- Unlock requests result in the request being deleted, and later requests are checked to see if they can now be granted
- If transaction aborts, all waiting or granted requests of the transaction are deleted
 - lock manager may keep a list of locks held by each transaction, to implement this efficiently





Timestamp-Based Protocols

- Each transaction T_i is issued a timestamp $TS(T_i)$ when it enters the system.
 - Each transaction has a *unique* timestamp
 - Newer transactions have timestamps strictly greater than earlier ones
 - Timestamps can be implemented with
 - System clock value
 - logical counter
- Timestamp-based protocols manage concurrent execution such that
time-stamp order = serializability order



Timestamp-Ordering Protocol

The **timestamp ordering (TSO) protocol**

- Maintains for each data Q two timestamp values:
 - **W-timestamp**(Q) is the largest time-stamp of any transaction that executed **write**(Q) successfully.
 - **R-timestamp**(Q) is the largest time-stamp of any transaction that executed **read**(Q) successfully.
- These timestamps are updated whenever a new **read**(Q) or **write**(Q) instruction is executed.
- Imposes rules on read and write operations to ensure that
 - Any conflicting operations are executed in timestamp order
 - Out of order operations cause transaction rollback



Timestamp-Based Protocols (Cont.)

- Suppose a transaction T_i issues a **read**(Q)
 1. If $TS(T_i) < \text{W-timestamp}(Q)$, then T_i needs to read a value of Q that was already overwritten.
 - Hence, the **read** operation is rejected, and T_i is rolled back.
 2. If $TS(T_i) \geq \text{W-timestamp}(Q)$, then the **read** operation is executed, and $\text{R-timestamp}(Q)$ is set to $\max(\text{R-timestamp}(Q), TS(T_i))$.



Timestamp-Based Protocols (Cont.)

- Suppose that transaction T_i issues **write**(Q).
 1. If $TS(T_i) < R\text{-timestamp}(Q)$, then the value of Q that T_i is producing was needed previously, and the system assumed that that value would never be produced.
 - Hence, the **write** operation is rejected, and T_i is rolled back.
 2. If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i is attempting to write an obsolete value of Q .
 - Hence, this **write** operation is rejected, and T_i is rolled back.
 3. Otherwise, the **write** operation is executed, and $W\text{-timestamp}(Q)$ is set to $TS(T_i)$.

Example of Schedule Under TSO

- Is this schedule valid under TSO?

Assume that initially:

$$R\text{-TS}(A) = W\text{-TS}(A) = 0$$

$$R\text{-TS}(B) = W\text{-TS}(B) = 0$$

Assume $TS(T_{25}) = 25$ and
 $TS(T_{26}) = 26$

T_{25}	T_{26}
read(B)	read(B) $B := B - 50$ write(B)
read(A)	read(A)
display($A + B$)	$A := A + 50$ write(A) display($A + B$)

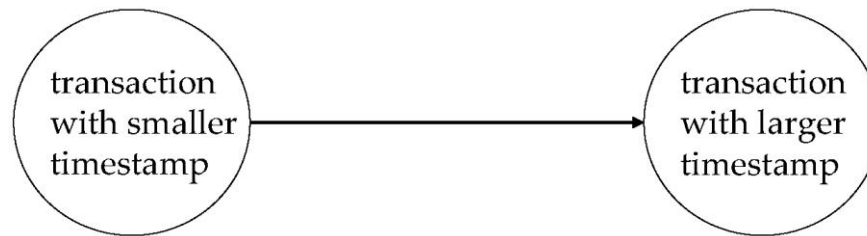
- How about this one,
 where initially
 $R\text{-TS}(Q) = W\text{-TS}(Q) = 0$

T_{27}	T_{28}
read(Q)	write(Q)
write(Q)	



Correctness of Timestamp-Ordering Protocol

- The timestamp-ordering protocol guarantees serializability since all the arcs in the precedence graph are of the form:



Thus, there will be no cycles in the precedence graph

- Timestamp protocol ensures freedom from deadlock as no transaction ever waits.
- But the schedule may not be cascade-free, and may not even be recoverable.



Recoverability and Cascade Freedom

- Solution 1:
 - A transaction is structured such that its writes are all performed at the end of its processing
 - All writes of a transaction form an atomic action; no transaction may execute while a transaction is being written
 - A transaction that aborts is restarted with a new timestamp
- Solution 2:
 - Limited form of locking: wait for data to be committed before reading it
- Solution 3:
 - Use commit dependencies to ensure recoverability



Snapshot Isolation

- **Motivation:** Decision support queries that read large amounts of data have concurrency conflicts with OLTP transactions that update a few rows
 - Poor performance results
- **Solution:** Snapshot isolation level
- A transaction is given a “snapshot” of the database at the time when it begins its execution.
- Transaction operates on the snapshot in complete isolation from concurrent transactions.



Snapshot Isolation

- Transactions are given two time stamps:
 - $StartTS(T_i)$: time at which T_i started
 - $CommitTS(T_i)$:time at which T_i requested validation
- Updates performed by a transaction must be validated before the transaction is allowed to commit.
- Snapshot isolation is based on multiversioning
 - each transaction that updates a data item creates a version of the data item



Snapshot Isolation

- Versions have only one timestamp- write time stamp, set to $CommitTS(T_i)$ if it is created by T_i
- When T_i reads a data item, the latest version of data item whose timestamp $\leq StartTS(T_i)$ is returned.
 - T_i does not see the updates of any transactions that committed after T_i started.
- **Update transactions – validation**-whether a transaction should be allowed to commit
 - Concurrent transactions operate in isolation, both trying to update the same data item, neither of them sees the update made by the other.



Snapshot Isolation

- If both transactions are allowed to commit, first update overwritten by the second – **lost update**
- Variants of snapshot isolation that prevents lost update
 - **First committer wins**
 - **First updater wins**
- If $StartTS(T_i) \leq StartTS(T_j) \leq CommitTS(T_i)$ or $StartTS(T_j) \leq StartTS(T_i) \leq CommitTS(T_j)$ then **T_j is concurrent with T_i**



Snapshot Isolation

- **First committer wins**
 - For each data item d that T_i intends to write, if there is a version of d whose time stamp is between $\text{StartTS}(T_i)$ and $\text{CommitTS}(T_i)$ then T_i aborts.
 - If no such data item is found, then T_i commits and its updates are written to the database
- **First updater wins**
 - Locking mechanisms is applied to updates
 - Transaction requests write lock to update a data item
 - If the lock is not held by a concurrent transaction, then lock is acquired



Snapshot Isolation

- If the item has been updated by any concurrent transaction, then T_i aborts.
- Otherwise T_i may proceed with its execution, including possibly committing
- If the write lock is already held by another other transaction T_j
 - T_i waits until T_j aborts or commits
 - If T_j aborts, the lock is released and T_i can acquire the lock and it is checked if the data item is already updated by a concurrent transaction. If yes then T_i aborts otherwise T_i proceeds.
 - If T_j commits then T_i must abort.



Benefits of Snapshot Isolation

- Reads are *never* blocked,
 - and also don't block other txns activities
- Avoids several anomalies
 - No dirty read, i.e. no read of uncommitted data
 - No lost update
 - I.e., update made by a transaction is overwritten by another transaction that did not see the update)
 - No non-repeatable read
 - I.e., if read is executed again, it will see the same value
- Problems with Snapshot Isolation
 - SI does not always give serializable executions
 - Serializable: among two concurrent txns, one sees the effects of the other
 - In SI: neither sees the effects of the other
 - Result: Integrity constraints can be violated



Snapshot Isolation

- Example of problem with SI

- Initially $A = 3$ and $B = 17$

- Serial execution: $A = ??$, $B = ??$
 - if both transactions start at the same time, with snapshot isolation: $A = ??$, $B = ??$

T_i	T_j
read(A)	read(A) read(B)
read(B)	
$A=B$	$B=A$
write(A)	
	write(B)

- Called **skew write**

- SI breaks serializability when transactions modify *different* items, each based on a previous state of the item the other modified
- Another issue with SI is that integrity constraints cannot be checked on snapshot-usually done outside of snapshot.



Serializable Snapshot Isolation

- **Serializable snapshot isolation (SSI)**: extension of snapshot isolation that ensures serializability
- Snapshot isolation tracks write-write conflicts, but does not track read-write conflicts
 - Where T_i writes a data item Q , T_j reads an earlier version of Q , but T_j is serialized after T_i
- Idea: track read-write dependencies separately, and roll-back transactions where cycles can occur
 - Ensures serializability



Summary

- We discussed the following:
 - Concurrency control protocol-Locking protocol
 - Two phase locking protocol (2PLP)- allows a transaction to lock a new data item only if that transaction has not yet unlocked any data item.
 - 2 PLP guarantees serializability but is not deadlock free.
 - Strict 2PLP and Rigorous 2PLP ensures recoverability and cascadelessness.



Summary

- We discussed the following
 - Timestamp-ordering protocol ensures serializability by selecting an ordering in advance between every pair of transactions.
 - A unique fixed timestamp is associated with each transaction and the timestamps determine the serializability order.
 - Snapshot isolation is a multiversion concurrency control protocol based on validation
 - Snapshot isolation does not guarantee serializability.
 - Extension of SI, serializable SI guarantees serializability.